Cletus: Section 1, Day 1

- Date: [YYYY-MM-DD]
- Course: Week 1 Day 1, running a first LLM locally, setting up the environment, and building a website summarizer
- Lab notebook: week1/day1.ipynb (the free Ollama version is in week1/solutions/day1_with_ollama.ipynb)
- Time spent: [e.g. 2h]

Key concepts

- LLM: a model that predicts the next bit of text (a token) based on the text before it.
- Ollama: lets you run open-source models on your own laptop for free. `ollama run llama3.2` starts one. Use `llama3.2:1b` if your machine is small, and skip `llama3.3` since at 70B it's way too big.
- Frontier models: the biggest closed models like GPT, Claude and Gemini. You reach them through an API and pay per use.
- API key and .env: the key goes in a `.env` file as `OPENAI_API_KEY=sk-proj-...` and gets loaded with `load_dotenv()`. The .env file should never be committed.
- System prompt: tells the model what its job is and what tone to use.
- User prompt: the actual question or content we want a response to.
- Messages: the API expects a list of dicts, each with a `role` (`system`, `user`, and later `assistant`) and the `content`.
- Chat Completions API: we send the messages and the reply comes back in `response.choices[0].message.content`.
- Summarization is one of the most common business uses: news, reports, CVs, reviews, meeting notes.

Code / examples

The Day 1 project cut down to the essentials:

```
from dotenv import load_dotenv
from openai import OpenAI
from scraper import fetch_website_contents  # uses requests and BeautifulSoup, returns the title
and text (first 2,000 chars)

load_dotenv(override=True)
openai = OpenAI()  # picks up OPENAI_API_KEY from the environment

system_prompt = "You summarize websites in markdown, ignoring navigation text."
user_prompt_prefix = "Here are the contents of a website. Provide a short summary:\n\n"

def summarize(url):
    website = fetch_website_contents(url)
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt_prefix + website},
    ]
```

```
    response = openai.chat.completions.create(model="gpt-4.1-mini", messages=messages)
    return response.choices[0].message.content

print(summarize("https://edwarddonner.com"))
```

The flow is: scrape the page, build the messages, call the model, read the reply.

Exercise / challenge

1. Change the tone in the system prompt (reply in Spanish, be formal, be funny) and compare the summaries.
2. Build a summarizer without a website. Paste an email or article into the user prompt and ask for 3 bullet points and a subject line.
3. Try `summarize("https://openai.com")`. It comes back almost empty. Why? (answer below)

Questions & blockers

Things to watch out for: - Sites built with JavaScript (React apps) or with bot protection return little or no text, because the scraper only reads the raw HTML. Selenium or Playwright can get around that. - A NameError in the notebook usually means the cells weren't run from the top. Also pick the `.venv` kernel for each notebook. - OpenAI keys start with `sk-proj-`. Make sure there are no spaces around the key in `.env`.

To discuss with my partner

- Where could we use summarization at work?
- What setup problems did each of us hit (Cursor, uv, kernel, API key)?

Resources

- Week 1 code: https://github.com/ed-donner/llm_engineering/tree/main/week1
- Setup guide: setup/SETUP-new.md, and setup/troubleshooting.ipynb if something breaks
- Using free models instead of paid APIs: guides/09_ai_apis_and_ollama.ipynb